

Hydra Minecraft - Project Close-out Report

Real-time, cross-server world synchronization through Cardano's Hydra Layer 2.

Name of project and Project URL on IdeaScale/Fund

Hydra Minecraft - a tech demo that uses Minecraft to showcase real-time, cross-server world synchronization through Cardano's Hydra Layer 2.

Project Catalyst URL:

<https://projectcatalyst.io/funds/14/cardano-use-cases-concepts/hydra-minecraft>

Your Project Number

1400061

Name of project manager

Adam Valach

Date project started

December 2025

Date project completed

June 2026

List of challenge KPIs and how the project addressed them

Hydra Minecraft was proposed to Catalyst as an innovative tech demo and open-source artifact to showcase Cardano's scaling technology in a way that's accessible to a broad audience. The project-level KPIs and how Hydra Minecraft addressed each one:

- **Demonstrate a novel use of Hydra / Cardano scaling tech.** Hydra Minecraft goes beyond a single-app speed demo and uses Hydra as a shared ledger between two independent Minecraft servers. Every block placed or broken on one server is committed as a Hydra transaction and replayed in real time on the other. This shows Hydra working as a synchronization backbone for shared interactive state, not just as a fast payment channel.
- **Produce an open-source, reusable artifact.** The entire project - plugin source, Docker Compose stack, Hydra head logs, and load test report - is published under the open-source repository at github.com/CardanoProphet/hydra-minecraft. The Docker setup in particular is reusable outside the gaming context: it fully automates wallet generation, Mithril-based preprod ledger bootstrapping, Cardano node sync, and opening a Hydra head, and can be lifted into any project that needs a turnkey Hydra environment.
- **Engage and educate the wider community.** Each milestone shipped with a written progress report and (for M1-M2) a video walkthrough, so the audience could follow not just the result but the engineering choices behind it. The final close-out video summarizes the project end-to-end for viewers who join late.
- **Build on existing scaling demos.** The proposal explicitly built on the visibility of "Doom on Hydra." Hydra Minecraft takes a complementary approach: instead of streaming frames, it

synchronizes discrete world events, which is far less transaction-heavy per player and therefore scales to many more concurrent participants - a different and useful data point for the ecosystem.

List of project KPIs and how the project addressed them

The project tracked three measures of success: the capacity of the system (how many players Hydra Minecraft can realistically support), community engagement around the progress posts and videos, and on-chain activity from real in-game events.

- **System capacity.** Measured through a rigorous load test driving transactions at maximum sustainable throughput across multiple parallel UTxO chains on both Hydra nodes. The result was measured running inside WSL2 + Docker on a single development machine with default Hydra parameters. This setup was intentional: the goal was to show that anyone can run Hydra Minecraft on consumer hardware with a single command, no server rack required. Docker provides exactly that - a one-command bootstrap that works on any machine. The two layers of virtualization (WSL2 + Docker) do impose significant overhead and represent a heavily bottlenecked baseline; on dedicated server hardware the numbers would be considerably higher. Both Minecraft servers and both Hydra nodes ran on the same machine in separate containers - so while the inter-node communication was local, the architecture was deliberately designed with one Hydra node per Minecraft server. In a real deployment, each server operator would run their own node, which mirrors how a healthy decentralized network should look rather than funneling everything through a shared bottleneck. A single-node shortcut would push raw numbers higher but would undermine that principle. Translated through community-shared block-breaking statistics, that capacity supports **~246 concurrent players at the highest real-world mining rate we could find** (0.79 blocks/sec, from a player who had broken over 8.7 million blocks lifetime), and **~630 concurrent players at the more typical pace** of 0.308 blocks/sec. For context, a Minecraft server is considered "very large" at 500 concurrent players; servers above 650 are exceptionally rare. The full report is at [LOAD_TEST_REPORT.md](#).
- **Open-source educational artifacts.** Addressed through three written milestone reports on Medium, two milestone video walkthroughs on YouTube, a close-out video, and the close-out report - each documenting not just the result but the engineering decisions behind it. The full repository, including plugin source, the Docker Compose stack, the Hydra head log, and the load test report, is public on GitHub with no proprietary dependencies, so the work is directly reusable by other Cardano builders.
- **On-chain activity demonstrating real-time scalability.** Addressed by the Hydra head log captured during Milestone 2 and the load test runs in Milestone 3. The Milestone 2 log file (committed at [log/ws.log](#)) contains real Hydra transactions submitted from in-game block events, with full event metadata (block type, coordinates, player, timestamp) embedded in each transaction. The load test runs further demonstrate sustained throughput at the rates described above.

Key achievements (in particular around collaboration and engagement)

- **Complete end-to-end synchronization loop delivered.** Two independent Minecraft servers, sharing nothing but a Hydra head, mirror each other's worlds in real time. Block events on one server are captured by the plugin, serialized into a Cardano transaction with full event metadata, submitted to the local Hydra node, observed by the second server over WebSocket, and replayed into its world - at sub-100ms latency under load.

- **Turnkey Hydra environment as a reusable artifact.** The Docker Compose stack automates everything from wallet generation through Mithril ledger snapshot download, Cardano node sync, Hydra head initialization/commit/open, and Minecraft server startup. After the initial preprod bootstrap, subsequent runs spin up in ~2 minutes. This is independently useful for any project that needs a Hydra dev environment without hand-rolling infrastructure.
- **Quantified scalability claim, not a vibe.** Rather than asserting "Hydra is fast," the project delivered a measured result with methodology: more than 630 concurrent players supported.
- **Transparent progress communication.** Three written milestone reports and two video reports were published openly during the build, so the community could follow the engineering choices in real time rather than only seeing the final artifact.
- **Open-source release.** All code, configuration, logs, and reports are public under github.com/CardanoProphet/hydra-minecraft with no proprietary dependencies.

Impact - measurable impact data

- **246 concurrent players supported at the highest real-world mining pace observed in community data; ~630 at the typical pace.** This places Hydra Minecraft's capacity comfortably above the "very large" Minecraft server threshold of 500 concurrent players.
- **Zero protocol fees per in-game action.** Every block placement and break is a real Cardano transaction settled on Hydra L2 at zero cost, demonstrating the economic profile required for high-frequency interactive applications.
- **Synchronization scope:** block placement and block breaking events, with full event metadata (block type, coordinates, player identity, timestamp) embedded in each transaction.
- **Bootstrap time:** ~10-15 minutes on first run (dominated by preprod ledger snapshot download), ~2 minutes on subsequent runs. This is a meaningful usability number for anyone trying to stand up a Hydra dev environment.
- **Artifacts produced and published:** 1 Minecraft plugin (Java, open-source), 1 Docker Compose stack, 1 load test report, 1 Hydra head transaction log, 3 written progress reports, 2 milestone video reports, 1 close-out video, 1 close-out report.

Why is this project important?

Hydra is often discussed in the abstract - a high-throughput, low-latency Layer 2 for Cardano. What Hydra Minecraft does is make that promise legible. Two Minecraft worlds, side by side on screen, mirroring each other in real time through nothing but a Hydra head, is something a non-engineer can grasp in seconds. And once you see it, the implication generalizes: any application where independent systems need a low-latency, cryptographically verifiable source of truth - multiplayer game shards, cross-service inventory, distributed simulation, on-chain smart contract-based anti-cheat analyzing events - can use the same pattern.

The community should care about this for three reasons. First, it's a strong outreach asset. The proposal explicitly aimed to do for cross-server state what "Doom on Hydra" did for single-app rendering, and the side-by-side demo is well-suited to a conference booth or a stage segment - particularly paired with Doom on Hydra to tell a more complete story about what Hydra can do today. Second, it's a credible scalability data point. Supporting hundreds of concurrent participants, was measured rather than claimed - and measured on an untuned development setup, which makes the production headline even stronger. Third, the byproducts are reusable. The Docker

stack is the fastest way I know of to get from "I have a fresh machine" to "I have an opened Hydra head on preprod," and it's useful well beyond Minecraft.

In short: Hydra Minecraft is the kind of demo that makes Hydra feel real to people who don't read whitepapers, and it leaves behind production-grade artifacts for the developers who do.

Key learnings

- **Discrete event synchronization scales better than frame streaming.** The proposal positioned Hydra Minecraft as a complement to "Doom on Hydra." Building it confirmed the core bet: synchronizing discrete world events (block place/break) rather than streaming rendered frames is dramatically less transaction-heavy per participant.
- **Parallel UTxO chains are essential for sustained throughput.** A single sequential UTxO chain serializes transactions and caps throughput at the round-trip latency of one transaction at a time. Driving multiple parallel UTxO chains on each Hydra node was what unlocked maximum sustainable throughput, and proved to be the single most important factor in the load-test results.
- **One Hydra node per server is the right topology, even though it costs raw numbers.** Running a node per Minecraft server mirrors how a decentralized network should actually look, with each operator running their own infrastructure. A single shared node would have produced bigger headline numbers but would have misrepresented the architecture. Choosing the honest topology over the flattering one was a deliberate and worthwhile trade-off.
- **Virtualization overhead dominates the baseline - and that's worth stating plainly.** Measuring inside WSL2 + Docker on one consumer machine imposes significant overhead, so the reported capacity is a lower bound rather than a ceiling. The upside is reproducibility: anyone can run the whole stack with a single command, which we judged more valuable for a public demo than a tuned-but-unrepeatable benchmark.
- **Translating throughput into a human metric makes the result legible.** Raw transactions-per-second means little to a non-engineer. Converting sustained throughput into "concurrent players" using community-shared block-breaking statistics turned an abstract benchmark into a claim anyone can reason about, and was key to making the demo communicate.
- **First-run bootstrap is dominated by ledger sync, not by our code.** Standing up a Hydra head on preprod is gated almost entirely by downloading and replaying the ledger snapshot. Using a Mithril-based snapshot cut first-run time to ~10-15 minutes and subsequent runs to ~2 minutes - a usability lesson that applies to any Hydra dev environment, not just this one.

Next steps for the project

The funded scope - the plugin, the Docker Compose stack, the Hydra head log, the load test, and the reports - is complete. Possible directions for follow-up work:

- **Expand the synchronization scope.** Extend beyond block placement and breaking to other world state - entities, inventories, redstone, and chat - to demonstrate richer shared-world use cases on the same Hydra backbone.
- **Scale beyond two servers.** Explore more than two Minecraft servers participating in a single Hydra head, or a mesh of heads, to test how the synchronization pattern behaves as the network grows.
- **Use it as an outreach asset.** Pair the side-by-side demo with "Doom on Hydra" for conference booths and stage segments to tell a fuller story about what Hydra can do today.

Final thoughts/comments

We are grateful to Project Catalyst and the voting community for funding this work. Hydra is usually discussed in the abstract - a high-throughput, low-latency Layer 2 - and the goal here was to make that promise into something a non-engineer can grasp in seconds: two Minecraft worlds, side by side, mirroring each other through nothing but a Hydra head.

We believe the most valuable outcome is not only the demo itself but the reusable artifacts it leaves behind - in particular the turnkey Docker stack that takes a fresh machine to an opened Hydra head on preprod with a single command, which is useful well beyond Minecraft. The team intends to keep developing Hydra Minecraft and welcomes collaboration from anyone interested in shared, low-latency, cryptographically verifiable state on Cardano.

Links to other relevant project sources or documents

- **Source repository (plugin, Docker stack, logs, reports):**
<https://github.com/CardanoProphet/hydra-minecraft>
- **Plugin source:** <https://github.com/CardanoProphet/hydra-minecraft/tree/main/hydramc>
- **Docker Compose setup:** <https://github.com/CardanoProphet/hydra-minecraft/tree/main/docker>
- **Load test report:**
https://github.com/CardanoProphet/hydra-minecraft/blob/main/LOAD_TEST_REPORT.md
- **Milestone 1 written progress report (Medium):**
<https://medium.com/@cardanoprophet/hydra-minecraft-progress-report-1-862bac9a207e>
- **Milestone 2 written progress report (Medium):**
<https://medium.com/@cardanoprophet/hydra-minecraft-progress-report-2-15271286c6ed>
- **Milestone 3 written progress report (Medium):**
<https://medium.com/@cardanoprophet/hydra-minecraft-progress-report-3-f2208130058c>
- **Milestone 1 video report (YouTube):** <https://youtu.be/ZoBKqf6Eobw>
- **Milestone 2 video report (YouTube):** <https://youtu.be/5bS4a3MLPzE>
- **Milestone 2 Hydra head transaction log:**
<https://github.com/CardanoProphet/hydra-minecraft/blob/main/log/ws.log>

Link to Close-out video

https://youtu.be/X4PbCKd_TBU